Spark高级编程 (II)



摘要

- Spark Streaming
- Spark Structured Streaming
- GraphX



摘要

- Spark Streaming
- Spark Structured Streaming
- GraphX



Spark Streaming

Spark Streaming makes it easy to build scalable fault-tolerant streaming applications.

Ease of Use

Build applications through high-level operators.

Spark Streaming brings Apache Spark's language-integrated API to stream processing, letting you write streaming jobs the same way you write batch jobs. It supports Java, Scala and Python.

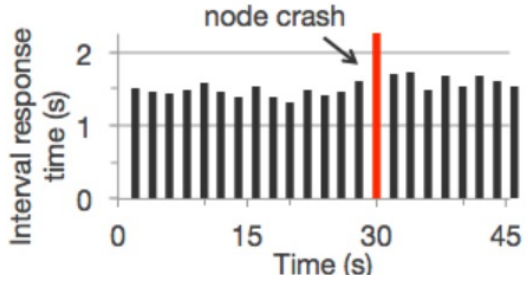
```
TwitterUtils.createStream(...)
    .filter(_.getText.contains("Spark"))
    .countByWindow(Seconds(5))
```

Counting tweets on a sliding window

Fault Tolerance

Stateful exactly-once semantics out of the box.

Spark Streaming recovers both lost work and operator state (e.g. sliding windows) out of the box, without any extra code on your part.



Spark Integration

Combine streaming with batch and interactive queries.

By running on Spark, Spark Streaming lets you reuse the same code for batch processing, join streams against historical data, or run ad-hoc queries on stream state. Build powerful interactive applications, not just analytics.

```
stream.join(historicCounts).filter {
  case (word, (curCount, oldCount)) =>
    curCount > oldCount
}
```

Find words with higher frequency than historic data



架构


Kafka
Flume
HDFS
ZeroMQ
Twitter
Spark Streaming
HDFS
Databases
Dashboards


input data stream
Spark Streaming
batches of input data
Spark Engine
batches of processed data

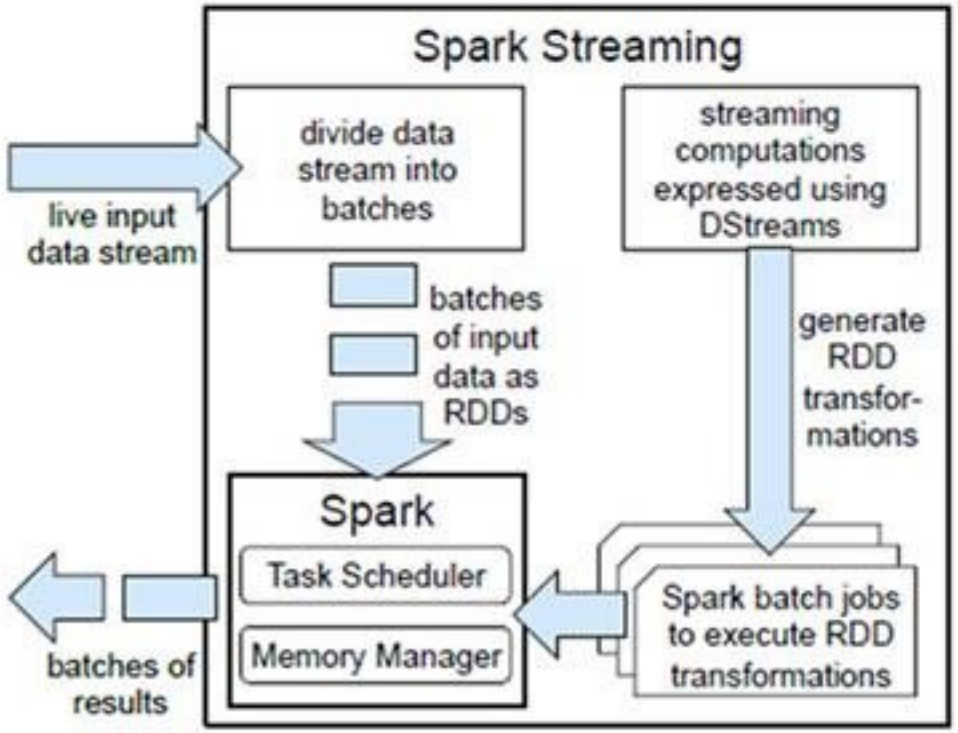


工作原理

- Spark Streaming将流式计算分解成一系列短小的批处理作业，具有如下特性：
  - 能线性扩展至超过数百个节点；
  - 实现亚秒级延迟处理；
  - 可与Spark批处理和交互式处理无缝集成；
  - 提供了一个简单的API实现复杂的算法；
  - 更多的网络流方式支持，包括Kafka、Flume、Kinesis、Twitter、ZeroMQ等。

DStream抽象

- DStream (Discretized Stream, 离散流)：连续的数据流，由一系列RDDs组成。





DStream抽象

- DStream的核心思想是将计算作为一系列较小时间间隔的、状态无关的、确定批次的任务，每个时间间隔内接收到的输入数据被可靠地存储在集群中，作为它的一个输入数据集。当某个时间间隔完成，将对相应的数据集并行地进行Map、Reduce和groupBy等操作，产生中间数据或输出新的数据集，并存储在RDD中。任务间的状态可以通过RDD重新计算，得益于计算任务被分解成一系列的小任务，用户可以在合适的粒度上呈现任务间的依赖关系。



DStream抽象

- 两类操作：
  - 转换操作：生成一个新的DStream
  - 输出操作：把数据写入外部系统中
- 增加了与时间相关的新操作，比如滑动窗口



简单的例子

- 例：从监听TCP套接字的数据服务器获取文本数据，然后计算文本中包含的单词数。

```
import org.apache.spark._
import org.apache.spark.streaming._
import org.apache.spark.streaming.StreamingContext._
// Create a local StreamingContext with two working thread and batch interval of 1 sec
ond
val conf = new SparkConf().setMaster("local[2]").setAppName("NetworkWordCount")
val ssc = new StreamingContext(conf, Seconds(1))
```

```
// Create a DStream that will connect to hostname:port, like localhost:9999
val lines = ssc.socketTextStream("localhost", 9999)
```

```
// Split each line into words
val words = lines.flatMap(_.split(" "))
```

简单的例子

- 例：从监听TCP套接字的数据服务器获取文本数据，然后计算文本中包含的单词数。

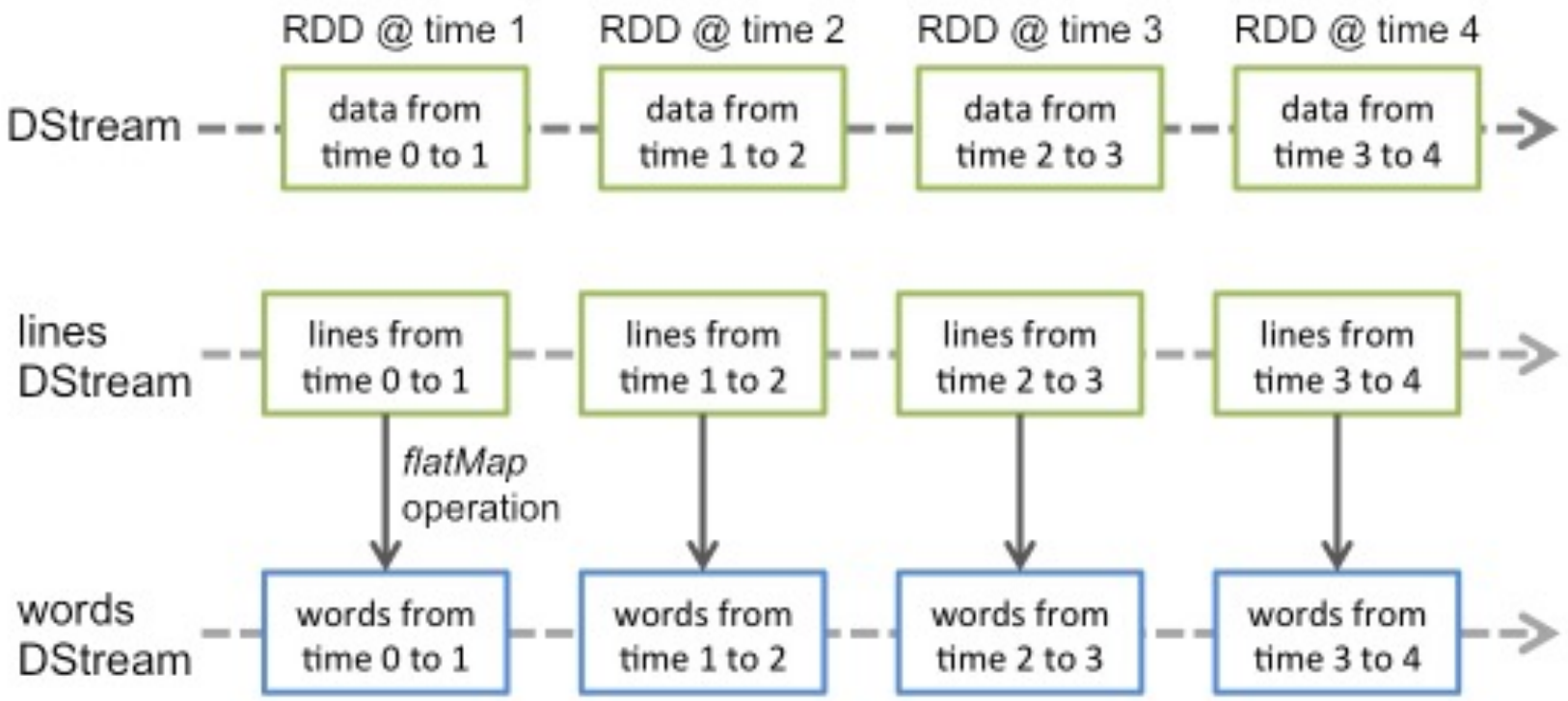
```
import org.apache.spark.streaming.StreamingContext._
// Count each word in each batch
val pairs = words.map(word => (word, 1))
val wordCounts = pairs.reduceByKey(_ + _)
// Print the first ten elements of each RDD generated in this DStream to the console
wordCounts.print()
```

```
ssc.start()             // Start the computation
ssc.awaitTermination()  // Wait for the computation to terminate
```



抽象

- DStream抽象





输入源

- 每一个输入流DStream和一个Receiver对象关联，这个Receiver从源中获取数据，并将数据存入内存中进行处理。
- 输入源
  - 基本源：在StreamingContext API中直接使用，例如文件系统、套接字连接、Akka的actor等。
  - 高级源：包括Kafka，Flume，Twitter等，需要通过额外的类来使用。
- 多个数据流→多个Receiver，要分配足够的核（如果是本地运行，那么是线程）用以处理接收的数据并且运行receiver是非常重要的



DStream操作

- 转换操作：允许在DStream运行任何RDD-to-RDD函数，比如map, flatMap, filter, reduce, join等
- 状态操作
  - updateStateByKey：不断用新信息更新它的同时保持任意状态
  - 窗口操作：允许在一个滑动窗口数据上应用transformation算子，需制定两个参数：窗口长度（窗口的持续时间）和滑动的时间间隔（窗口操作执行的时间间隔）

```
// Reduce last 30 seconds of data, every 10 seconds
val windowedWordCounts = pairs.reduceByKeyAndWindow((a:Int,b:Int) => (a + b), Seconds(
30), Seconds(10))
```



DStream操作

Transformation	Meaning
map	对传入的每个元素，返回一个新的元素
flatMap	对传入的每个元素，返回一个或多个元素
filter	对传入的元素返回true或false，返回的false的元素被过滤掉
union	将两个DStream进行合并
count	返回元素的个数
reduce	对所有values进行聚合
countByValue	对元素按照值进行分组，对每个组进行计数，最后返回<K, V>的格式
reduceByKey	对key对应的values进行聚合
cogroup	对两个DStream进行连接操作，一个key连接起来的两个RDD的数据，都会以Iterable<V>的形式，出现在一个Tuple中。



DStream操作

Transformation	Meaning
join	对两个DStream进行join操作，每个连接起来的pair，作为新DStream的RDD的一个元素
transform	对数据进行转换操作
updateStateByKey	为每个key维护一份state，并进行更新（这个，我认为，是在普通的实时计算中，最有用的一种操作）
window	对滑动窗口数据执行操作（实时计算中最有特色的一种操作）



DStream操作

- 输出操作

Output Operation	Meaning
print()	在DStream的每个批数据中打印前10条元素，这个操作在开发和调试中都非常有用。在Python API中调用 `pprint()` 。
saveAsObjectFiles(prefix, [suffix])	保存DStream的内容为一个序列化的文件 `SequenceFile` 。每一个批间隔的文件的文件名基于 `prefix` 和 `suffix` 生成。"prefix-TIME_IN_MS[.suffix]"，在Python API中不可用。
saveAsTextFiles(prefix, [suffix])	保存DStream的内容为一个文本文件。每一个批间隔的文件的文件名基于 `prefix` 和 `suffix` 生成。"prefix-TIME_IN_MS[.suffix]"
saveAsHadoopFiles(prefix, [suffix])	保存DStream的内容为一个hadoop文件。每一个批间隔的文件的文件名基于 `prefix` 和 `suffix` 生成。"prefix-TIME_IN_MS[.suffix]"，在Python API中不可用。
foreachRDD(func)	在从流中生成的每个RDD上应用函数 `func` 的最通用的输出操作。这个函数应该推送每个RDD的数据到外部系统，例如保存RDD到文件或者通过网络写到数据库中。需要注意的是， `func` 函数在驱动程序中执行，并且通常都有RDD action在里面推动RDD流的计算。



DStream操作

- 缓存及持久化
  - persist()
  - DStream的持久化策略是将数据序列化在内存中。
  - 基于窗口或状态的操作，如reduceByWindow、reduceByKeyAndWindow和updateStateByKey，Dstream都会自动持久化在内存中，无须显式调用persist()方法。
  - 通过网络接收的流数据默认采取保存两份序列化后的数据在两个不同的节点上的持久化策略，从而实现容错。



Checkpointing机制

- Metadata checkpointing：保存流计算的定义信息到容错存储系统如HDFS中。这用来恢复应用程序中运行worker的节点的故障。元数据包括：
  - Configuration
  - DStream operations
  - Incomplete batches
- Data checkpointing：保存生成的RDD到可靠的存储系统中，这在有状态transformation（如结合跨多个批次的数据）中是必须的。有状态的transformation的中间RDD会定时存储到可靠存储系统中。



Checkpointing配置

- 在容错、可靠的文件系统（HDFS、S3等）中设置一个目录用于保存checkpoint信息。
  - streamingContext.checkpoint(checkpointDirectory)

```
// Function to create and setup a new StreamingContext
def functionToCreateContext(): StreamingContext = {
    val ssc = new StreamingContext(...)   // new context
    val lines = ssc.socketTextStream(...) // create DStreams
    ...
    ssc.checkpoint(checkpointDirectory)   // set checkpoint directory
    ssc
}

// Get StreamingContext from checkpoint data or create a new one
val context = StreamingContext.getOrCreate(checkpointDirectory, functionToCreateContex
t _)

// Do additional setup on context that needs to be done,
// irrespective of whether it is being started or restarted
context. ...

// Start the context
context.start()
context.awaitTermination()
```



Spark Streaming编程

- 首先创建StreamingContext：

```
#方法一：
val conf = new SparkConf().setAppName(appName).setMaster(master);
val ssc = new StreamingContext(conf, Seconds(1));
#方法二：可以使用已有的SparkContext来创建
val sc = new SparkContext(conf);
val ssc = new StreamingContext(sc, Seconds(1));
```

- 注：appName，是用来在Spark UI上显示的应用名称；master，是一个Spark、Mesos或者Yarn集群的URL，或者是local[*]；batch interval可以根据你的应用程序的延迟要求以及可用的集群资源情况来设置。



Spark Streaming编程

- 接下来的流程：
  1. 通过创建输入DStream来创建输入数据源。
  2. 通过对DStream定义transformation和output算子操作，来定义实时计算逻辑。
  3. 调用StreamingContext的start()方法，来开始实时处理数据。
  4. 调用StreamingContext的awaitTermination()方法，来等待应用程序的终止。可以使用CTRL+C手动停止，或者就是让它持续不断的运行进行计算。
  5. 也可以通过调用StreamingContext的stop()方法，来停止应用程序。



Spark Streaming编程

- 注意事项：
  1. 只要一个StreamingContext启动之后，就不能再往其中添加任何计算逻辑了。比如执行start()方法之后，还给某个DStream执行一个算子。
  2. 一个StreamingContext停止之后，是肯定不能够重启的，调用stop()之后，不能再调用start()。
  3. 一个JVM同时只能有一个StreamingContext启动，在你的应用程序中，不能创建两个StreamingContext。
  4. 调用stop()方法时，会同时停止内部的SparkContext，如果不希望如此，还希望后面继续使用SparkContext创建其他类型的Context，比如SQLContext，那么就用stop(false)。
  5. 一个SparkContext可以创建多个StreamingContext，只要上一个先用stop(false)停止，再创建下一个即可。



输入源

- 套接字

```
val lines = ssc.socketTextStream("localhost", 9999)
```

- 文件流

```
val logData = ssc.textFileStream(logDirectory)
val data = ssc.fileStream[KeyClass, ValueClass, InputFormatClass] (dataDirectory)
```

- 附加数据源
  - Apache Kafka
  - Twitter
  - Amazon Kinesis
  - Apache Flume

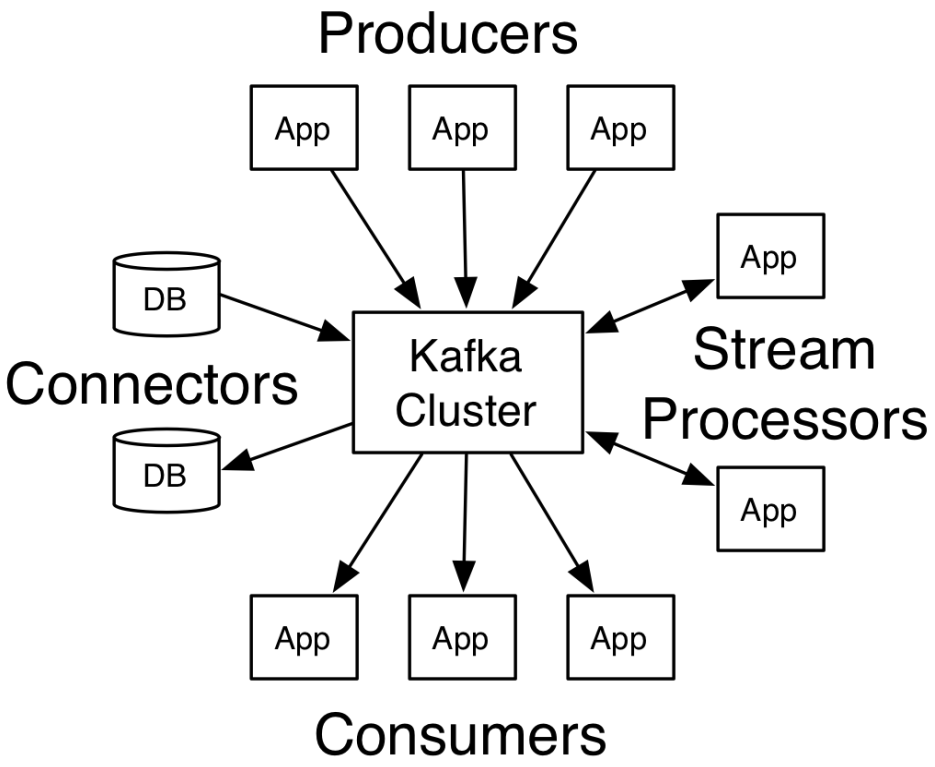


Apache Kafka

- Apache Kafka是一个分布式流处理平台。用于构建实时的数据管道和流式的app。它可以水平扩展，高可用，速度快，并且已经运行在数千家公司的生产环境。
- 流处理平台的三种特性
  - 可以让你发布和订阅流式的记录。这一方面与消息队列或者企业消息系统类似。
  - 可以储存流式的记录，并且有较好的容错性。
  - 可以在流式记录产生时就进行处理。



Apache Kafka





Apache Kafka

- Kafka维护按类区分的消息，称为主题（topic）
- 生产者（producer）向Kafka的主题发布消息
- 消费者（consumer）向主题注册，并且接收发布到这些主题的消息
- Kafka以一个拥有一台或多台服务器的集群运行着，每一台服务器称为broker
- 从高层来说，生产者（producer）通过网络发消息到Kafka集群，而Kafka集群则以下面这种方式对消费者进行服务。

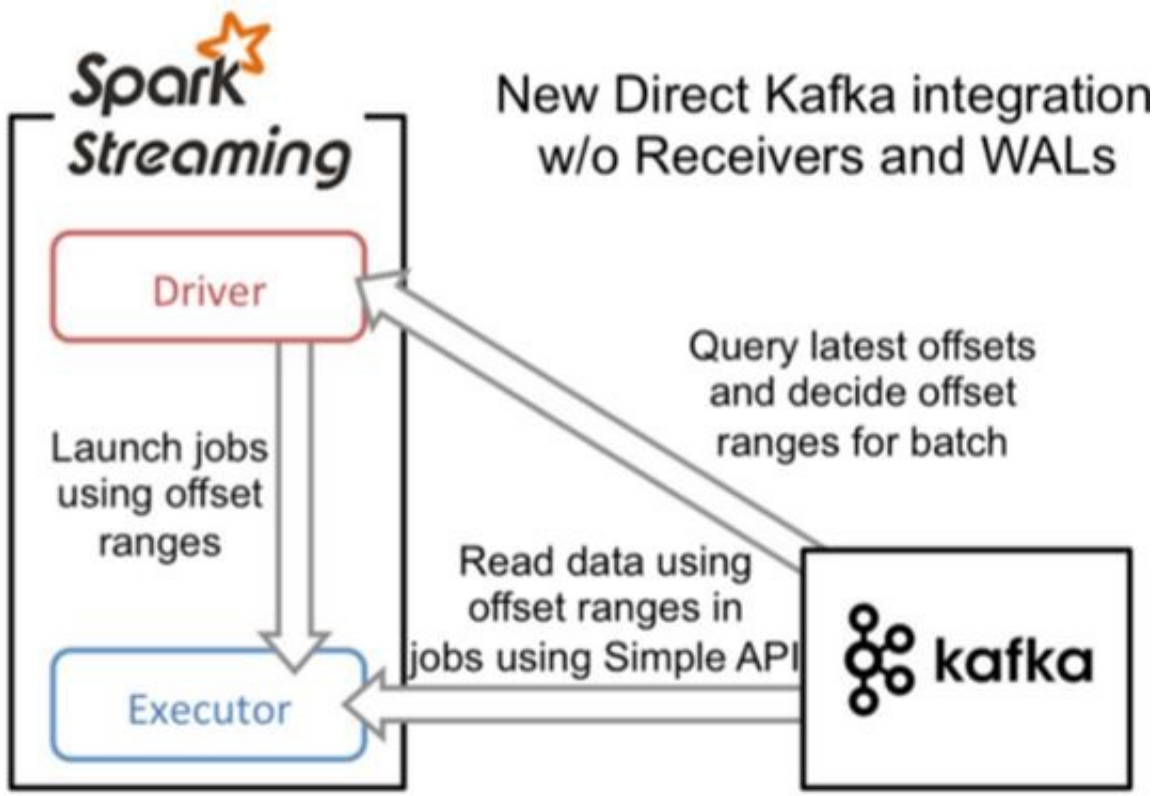


Apache Kafka

- 步骤
  1. 运行zookeeper服务器
  2. 运行kafka服务器
  3. 创建topic
  4. 查看topic是否存在
  5. 创建producer
  6. 创建consumer（测试）
  7. 提交Spark Streaming作业



Spark Streaming + Kafka





Spark Streaming + Kafka

- Linking
  - groupId = org.apache.spark
  - artifactId = spark-streaming-kafka-0-10_2.12
  - version = 3.0.1
- 编程
  - 通过KafkaUtils对象创建出Dstream；
  - 由于KafkaUtils可以订阅多个主题，因此它创建出的Dstream由成对的主题和消息组成；
  - 要创建一个流数据，需要使用StreamingContext实例，一个由逗号隔开的Zookeeper主机列表字符串、消费者组的名字（唯一名字），以及一个从主题到针对这个主题的接收器线程数的映射表来调用createStream()方法。



Spark Streaming + Kafka


```
…
Map<String, Integer> topicMap = new HashMap<>();
String[] topics = args[2].split(",");
for (String topic: topics) {  topicMap.put(topic, numThreads);  }

JavaPairReceiverInputDStream<String, String> messages =
        KafkaUtils.createStream(jssc, args[0], args[1], topicMap);

JavaDStream<String> lines = messages.map(Tuple2::_2);
JavaDStream<String> words = lines.flatMap(x ->  Arrays.asList (SPACE.split(x)).iterator());
JavaPairDStream<String, Integer> wordCounts = words.mapToPair(s -> new Tuple2<>(s,
1)).reduceByKey((i1, i2) -> i1 + i2);
```



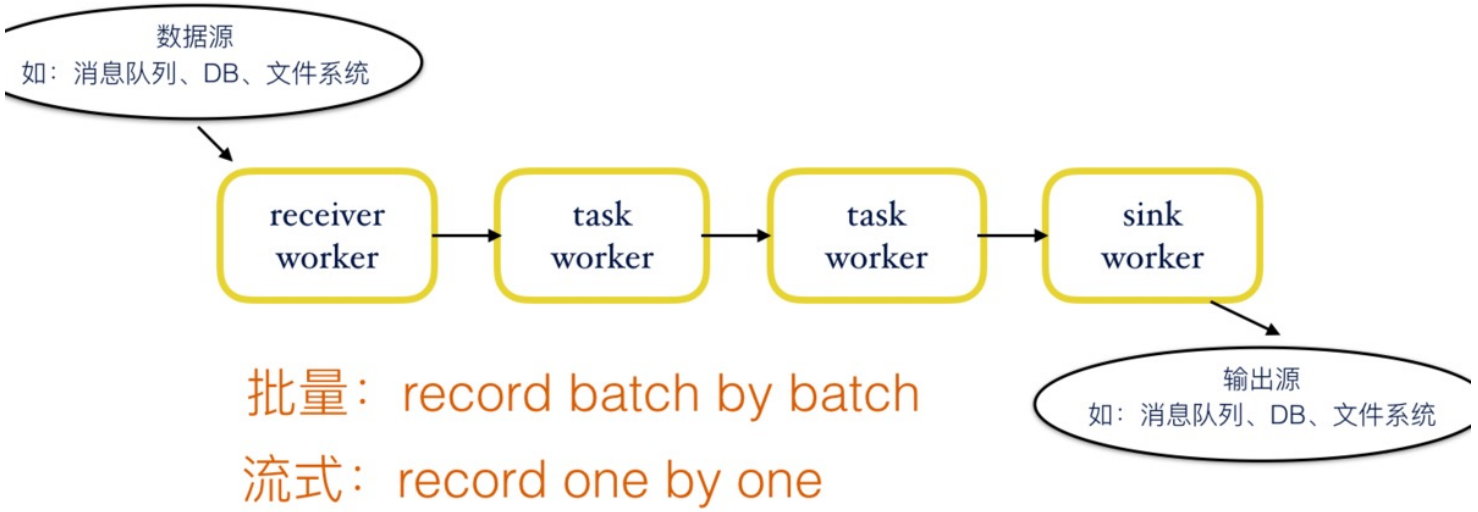
Spark Streaming + Kafka

- Usage: JavaKafkaWordCount <zkQuorum> <group> <topics> <numThreads>
  - <zkQuorum> is a list of one or more zookeeper servers that make quorum
  - <group> is the name of kafka consumer group
  - <topics> is a list of one or more kafka topics to consume from
  - <numThreads> is the number of threads the kafka consumer should use

- $ bin/run-example org.apache.spark.examples.streaming.JavaKafkaWordCount zoo01,zoo02, zoo03 my-consumer-group topic1,topic2 1

批量计算 vs 流式计算

- 批量和流式处理数据粒度不一样，批量每次处理一定大小的数据块（输入一般采用文件系统），一个task处理完一个数据块之后，才将处理好的中间数据发送给下游。流式计算则是以record为单位，task在处理完一条记录之后，立马发送给下游。





批量计算 vs 流式计算

- 区别
  - 数据处理单位
    - 批量计算按数据块来处理数据，每一个task接收一定大小的数据块
    - 流式计算的上游算子处理完一条数据后，会立马发送给下游算子，所以一条数据从进入流式系统到输出结果的时间间隔较短
  - 数据源
    - 批量计算通常处理的是有限数据（bound data），数据源一般采用文件系统，而流式计算通常处理无限数据（unbound data），一般采用消息队列作为数据源。
  - 任务类型
    - 批量计算中的每个任务都是短任务，任务在处理完其负责的数据后关闭，而流式计算往往是长任务，每个worker一直运行，持续接受数据源传过来的数据。



批量计算 vs 流式计算

- 离线=批量？实时=流式？
  - **离线和实时应该指的是：数据处理的延迟；批量和流式指的是：数据处理的方式。**两者并没有必然的关系。事实上Spark streaming就是采用小批量（batch）的方式来实现实时计算。

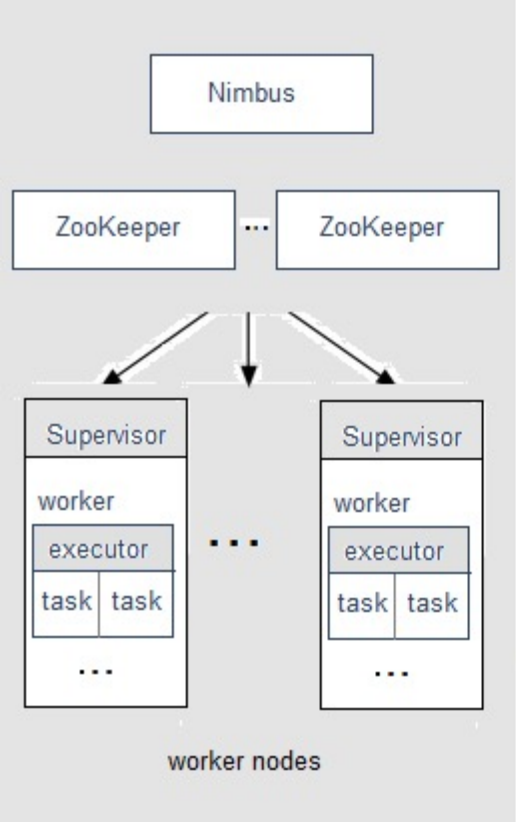


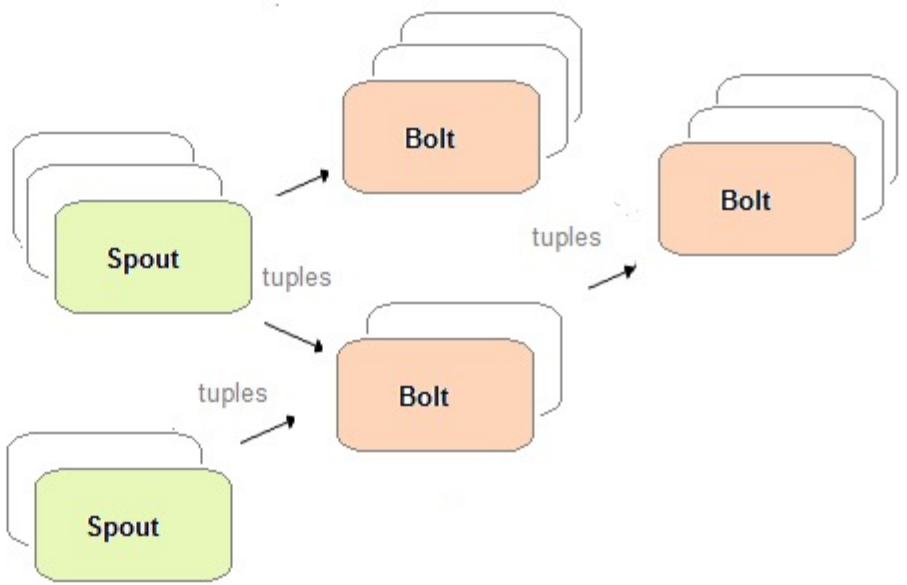
流式计算框架

- Apache Storm
- Apache Spark Streaming
- Apache Flink

流式计算框架

- Apache Storm



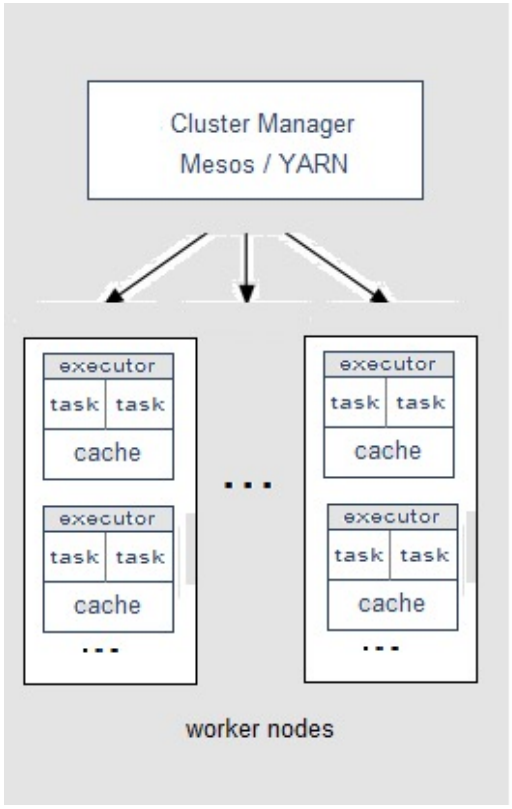


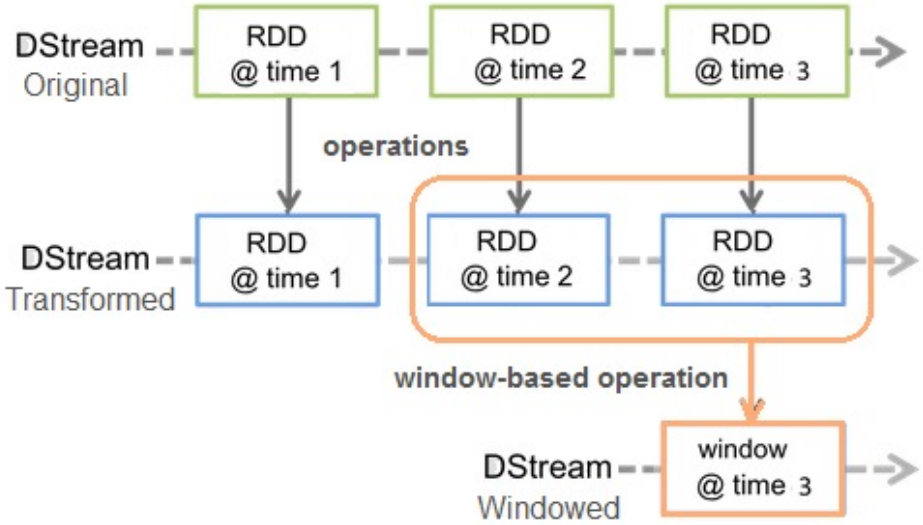
APACHE STORM™



流式计算框架

- Apache Spark Streaming

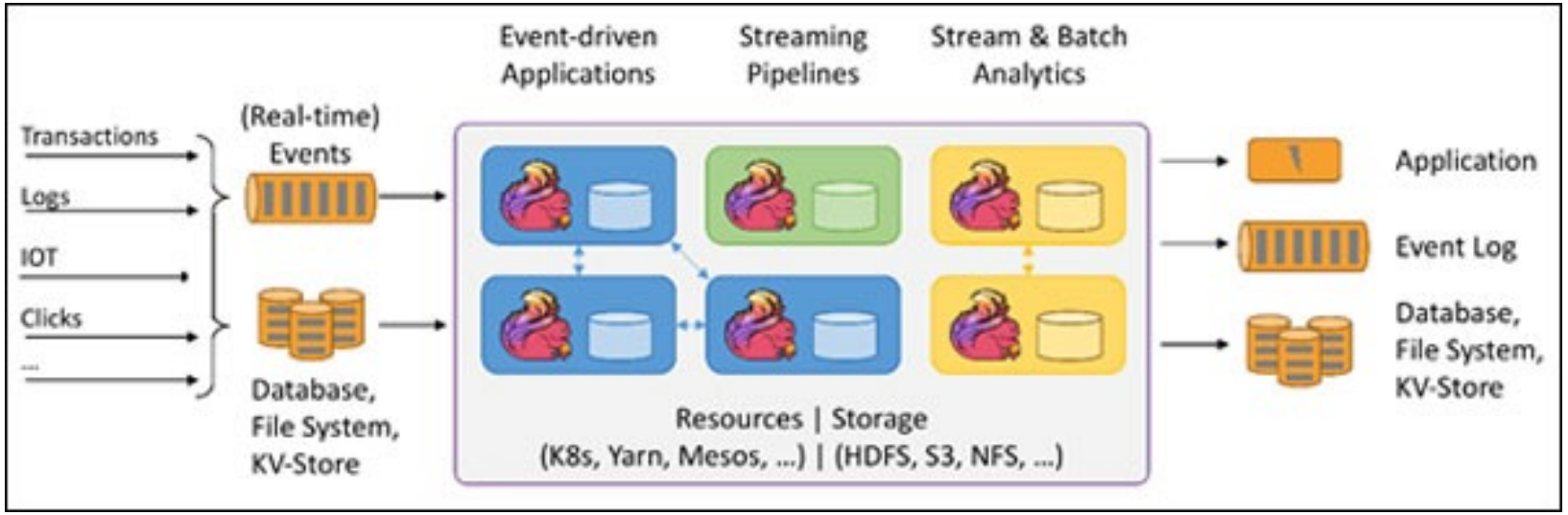






流式计算框架

- Apache Flink





Comparison

	Strom	Spark Streaming	Flink
Streaming Model	Native	Micro-batching	Native
Guarantees	At-Least-Once	Exactly-Once	Exactly-Once
Back Pressure	No	Yes	Yes
Latency	Very Low	Medium	Low
Throughput	Low	High	High
Fault Tolerance	Record ACKs	RDD Based Check Pointing	Check Pointing
Stateful	No	Yes (DStream)	Yes (Operators)



摘要

- Spark Streaming
- Spark Structured Streaming
- GraphX

Spark Structured Streaming

Spark Structured Streaming makes it easy to build streaming applications and pipelines with the same and familiar Spark APIs.

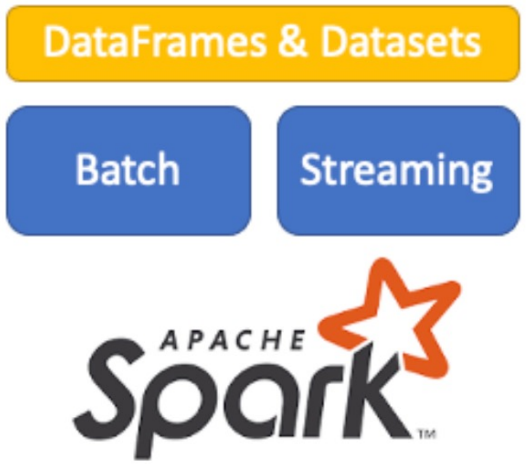
Easy to use

Spark Structured Streaming abstracts away complex streaming concepts such as incremental processing, checkpointing, and watermarks so that you can build streaming applications and pipelines without learning any new concepts or tools.

```
spark
  .readStream
  .select($"value".cast("string").alias("j
sonData"))
  .select(from_json($"jsonData",jsonSchema
).alias("payload"))
  .writeStream
  .trigger("1 seconds")
  .start()
```

Unified batch and streaming APIs

Spark Structured Streaming provides the same structured APIs (DataFrames and Datasets) as Spark so that you don't need to develop on or maintain two different technology stacks for batch and streaming. In addition, unified APIs make it easy to migrate your existing batch Spark jobs to streaming jobs.

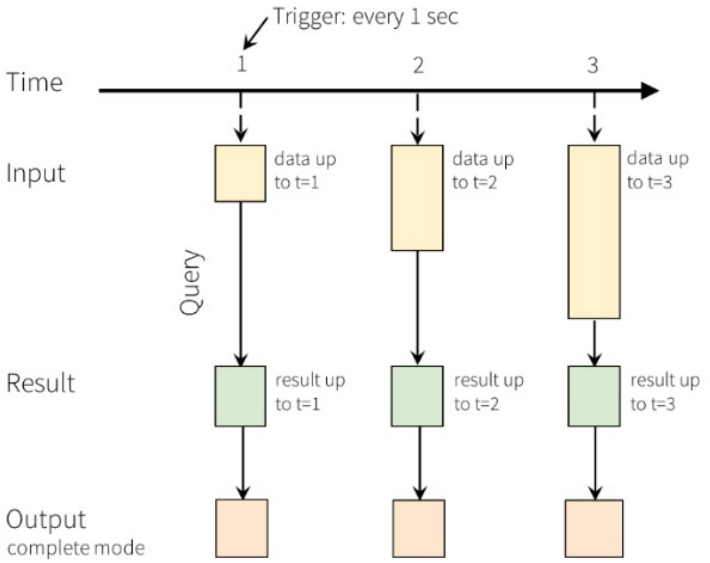




Spark Structured Streaming

Low latency and cost effective

Spark Structured Streaming uses the same underlying architecture as Spark so that you can take advantage of all the performance and cost optimizations built into the Spark engine. With Spark Structured Streaming, you can build low latency streaming applications and pipelines cost effectively.



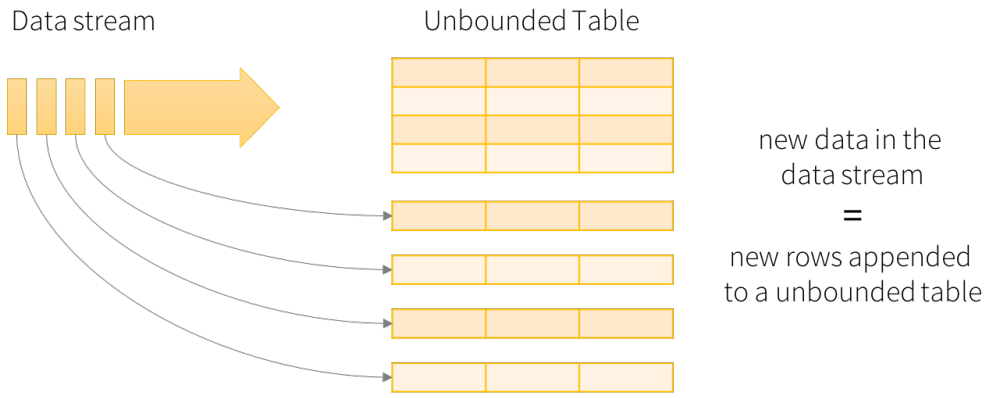


Spark Streaming vs. Structured Streaming

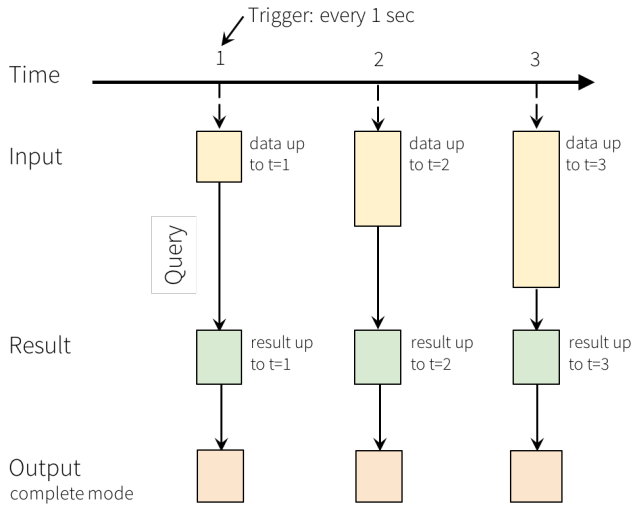
- Spark Streaming
  - Spark Streaming是Spark最初的流处理框架，使用了微批的形式来进行流处理。
  - 提供了基于RDDs的Dstream API，每个时间间隔内的数据为一个RDD，源源不断对RDD进行处理来实现流计算.
- Spark Structured Streaming
  - Spark 2.X出来的流框架，采用了无界表的概念，流数据相当于往一个表上不断追加行。
  - 基于Spark SQL引擎实现，可以使用大多数Spark SQL的function

区别

- 流模型
  - Spark Streaming采用微批的处理方法。每一个批处理间隔的为一个批，也就是一个RDD，我们对RDD进行操作就可以源源不断的接收、处理数据。
  - Structured Streaming将实时数据当做被连续追加的表。流上的每一条数据都类似于将一行新数据添加到表中。



Data stream as an unbounded table



Programming Model for Structured Streaming

每隔1秒从输入源获取数据到Input Table，并触发Query计算，然后将结果写入Result Table，之后根据指定的Output模式进行写出。



区别

- RDD、DataFrame、DataSet
  - Spark Streaming的DStream编程接口是RDD。

```
stream.foreachRDD(rdd => {
    balabala(rdd)
})
```

  - Structured Streaming使用DataFrame、DataSet的编程接口，处理数据时可以使用Spark SQL中提供的方法，数据的转换和输出会变得更加简单。

```
spark
    .readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", "hadoop01:9092")
    .option("subscribe", "order_data")
    .load()
    .select($"value".cast("string"))
    .as[String]
    .writeStream
    .outputMode("complete")
    .format("console")
```

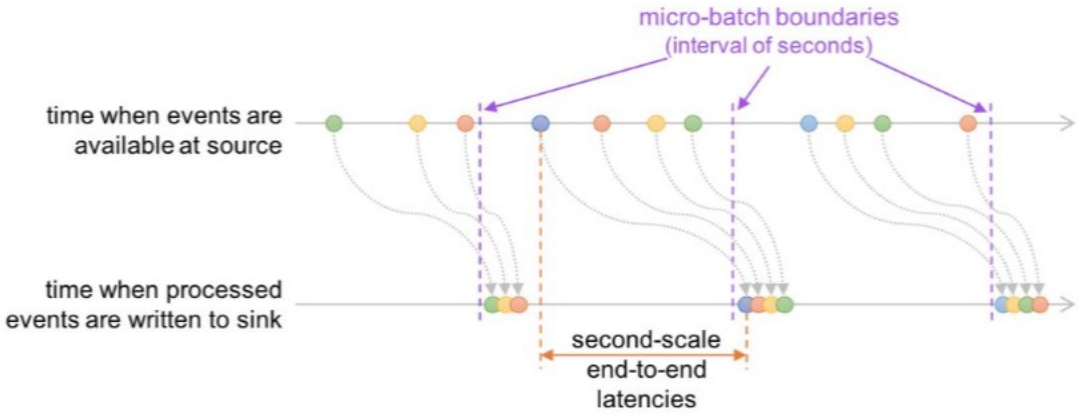


区别

- Process Time vs. Event Rime
  - Process Time：流处理引擎接收到数据的时间
  - Event Time：事件真正发生的时间
  - Spark Streaming由于其微批的概念，会将一段时间内接收的数据放入一个批内，进而对数据进行处理。划分批的时间是Process Time，而不是Event Time，Spark Streaming没有提供对Event Time的支持。
  - Structured Streaming提供了基于事件时间处理数据的功能，如果数据包含事件的时间戳，就可以基于事件时间进行处理。
- Structured Streaming的continuous mode是实时处理的，只要一有数据就会进行处理，时延基本在毫秒级别。

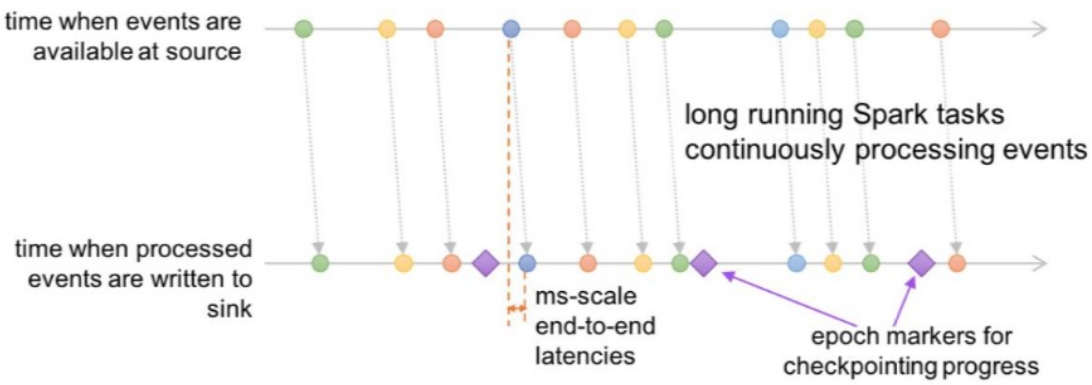
区别

Spark Streaming



Second-scale end-to-end latencies with Micro-batch Processing

Structured Streaming



Millisecond-scale end-to-end latencies with Continuous Processing



区别

- 可靠性保障
  - 两者在可靠性保证方面都是使用了checkpoint机制。checkpoint通过设置检查点，将数据保存到文件系统，在出现故障的时候进行数据恢复。
  - 在Spark Streaming中，如果我们需要修改流程序的代码，在修改代码重新提交任务时，是不能从checkpoint中恢复数据的（程序就跑不起来），是因为Spark不认识修改后的程序了。
  - 在Structured Streaming中，对于指定的代码修改操作，是不影响修改后从checkpoint中恢复数据的。



区别

- Sink：输出数据写入下游
  - Spark Streaming中提供了foreachRDD()方法，通过自己编程实现将每个批的数据写出。

```
stream.foreachRDD(rdd => { save(rdd) })
```

  - Structured Streaming自身提供了一些sink(Console Sink、File Sink、Kafka Sink等)，只要通过option配置就可以使用；对于需要自定义的Sink，提供了ForeachWriter的编程接口，实现相关方法就可以完成。

```
// console sink
val query = res.writeStream.outputMode("append").format("console").start()
```

- 总结：Structured Streaming有更简洁的API、更完善的流功能、更适用于流处理。而Spark Streaming更适用于偏批处理的场景。



Note

- Spark Streaming已成为legacy project，不再更新，推荐使用Spark Structured Streaming



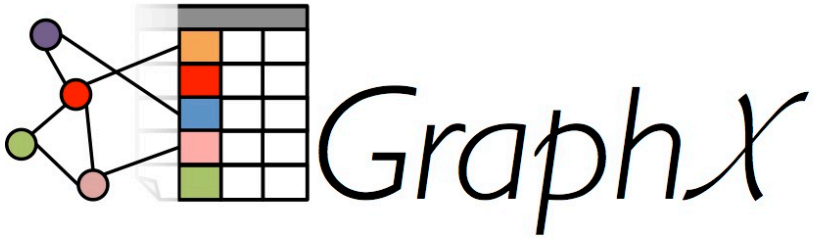
摘要

- Spark Streaming
- Spark Structured Streaming
- GraphX



GraphX

GraphX is Apache Spark's API for graphs and graph-parallel computation.



Flexibility

Seamlessly work with both graphs and collections.

GraphX unifies ETL, exploratory analysis, and iterative graph computation within a single system. You can view the same data as both graphs and collections, transform and join graphs with RDDs efficiently, and write custom iterative graph algorithms using the Pregel API.

```
graph = Graph(vertices, edges)
messages = spark.textFile("hdfs://...")
graph2 = graph.joinVertices(messages) {
  (id, vertex, msg) => ...
}
```

Using GraphX in Scala

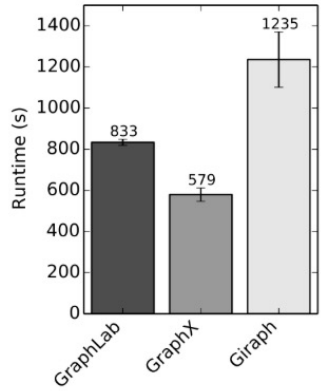


GraphX

Speed

Comparable performance to the fastest specialized graph processing systems.

GraphX competes on performance with the fastest graph systems while retaining Spark's flexibility, fault tolerance, and ease of use.



End-to-end PageRank performance
(20 iterations, 3.7B edges)

Algorithms

Choose from a growing library of graph algorithms.

In addition to a highly flexible API, GraphX comes with a variety of graph algorithms, many of which were contributed by our users.

- PageRank
- Connected components
- Label propagation
- SVD++
- Strongly connected components
- Triangle count

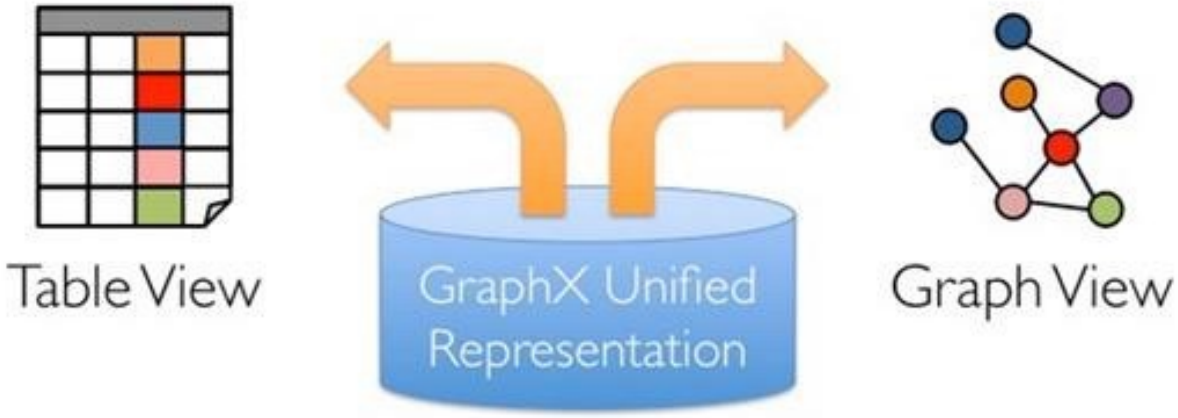


GraphX

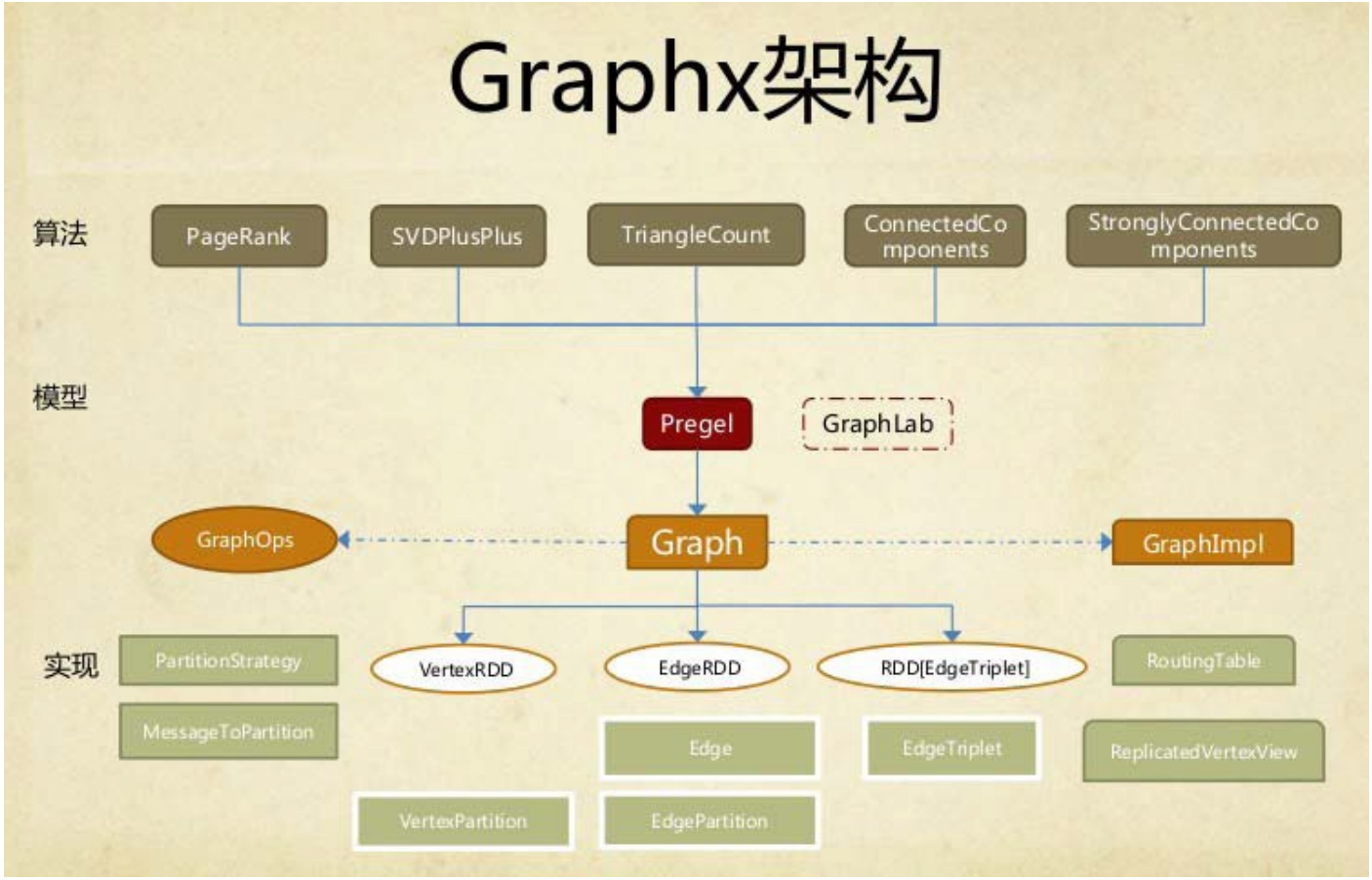
- GraphX是Spark中用于图和图并行计算的组件。
- GraphX通过扩展Spark RDD引入一个新的图抽象，一个将有效信息放在顶点和边的有向多重图。
- GraphX公开了一系列基本运算，以及一个优化后的Pregel API的变形。包括越来越多的图形计算和builder构造器，以简化图形分析任务。
- 在Spark之上提供了一站式解决方案，可以方便且高效地完成图计算的一整套流水作业。

GraphX核心抽象

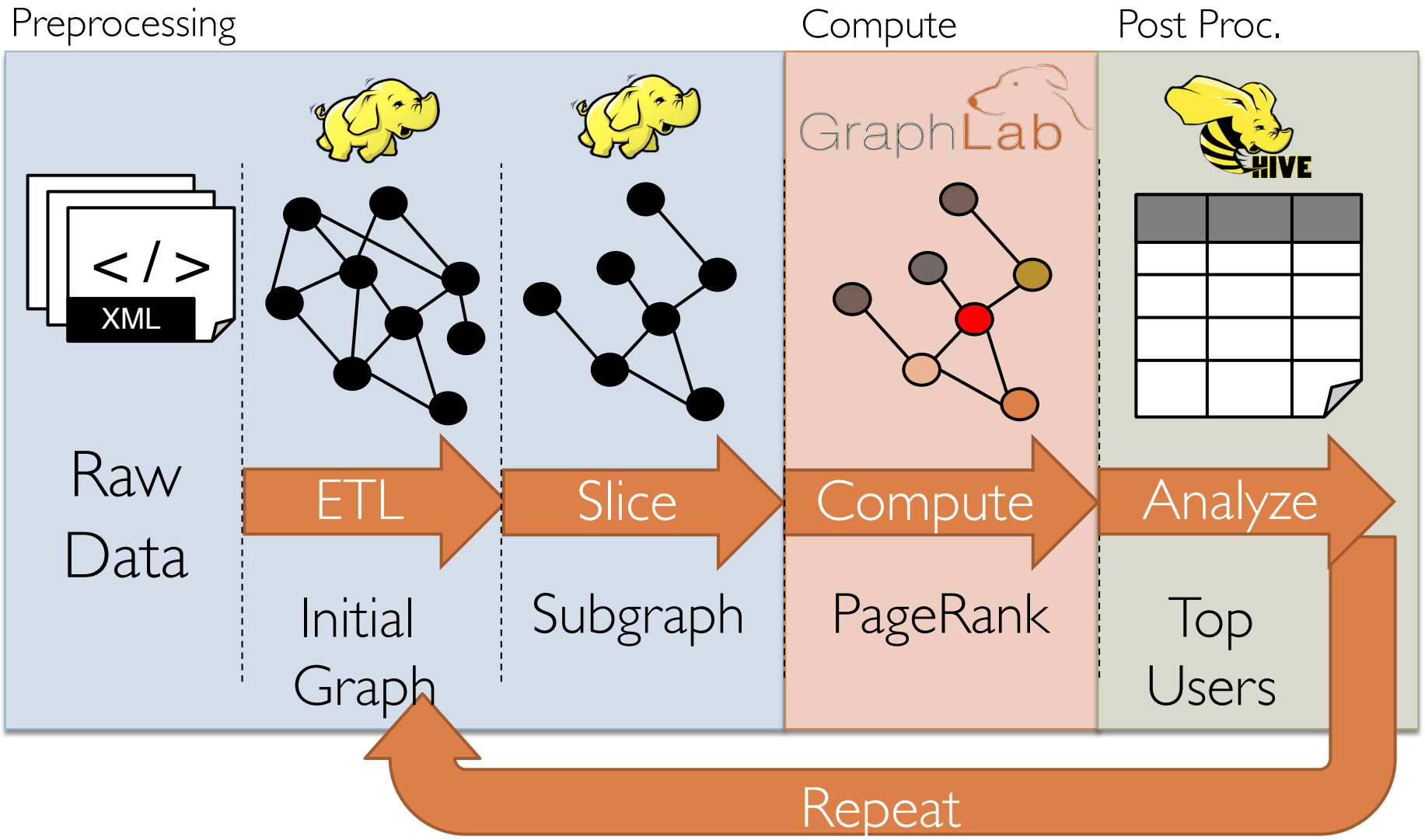
- 弹性分布式属性图（Resilient Distributed Property Graph），一种点和边都带属性的有向多重图。它扩展了Spark RDD的抽象，有Table和Graph两种视图，而只需要一份物理存储。两种视图都有自己独有的操作符，从而获得了灵活操作和执行效率。



GraphX框架


Graphx架构
算法
PageRank
SVDPlusPlus
TriangleCount
ConnectedComponents
StronglyConnectedComponents
模型
Pregel
GraphLab
GraphOps
Graph
GraphImpl
实现
PartitionStrategy
MessageToPartition
VertexRDD
EdgeRDD
RDD[EdgeTriplet]
RoutingTable
Edge
EdgeTriplet
ReplicatedVertexView
VertexPartition
EdgePartition

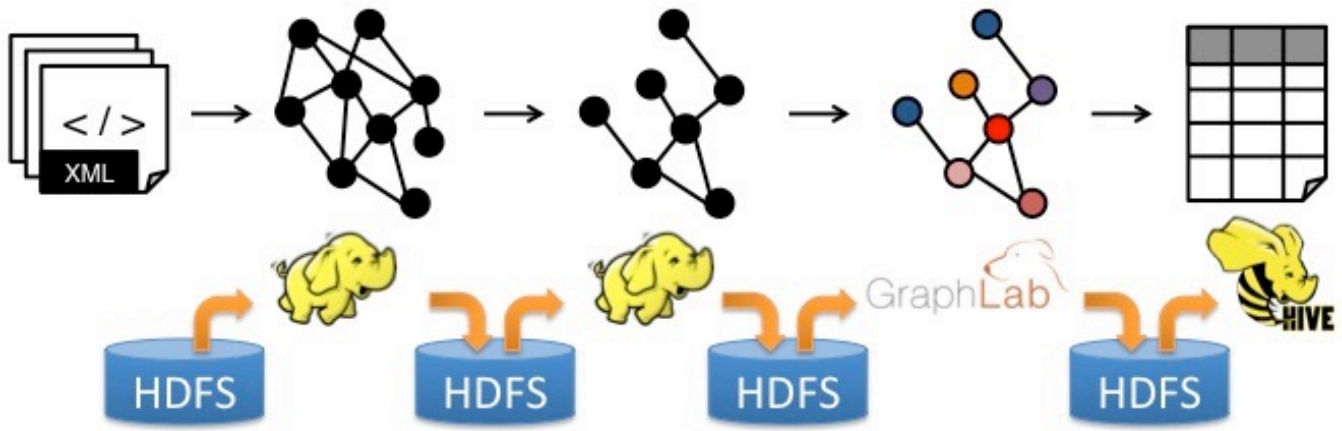
图处理流水线


Preprocessing
Compute
Post Proc.
GraphLab
HIVE
XML
Raw Data
ETL
Slice
Compute
Analyze
Initial Graph
Subgraph
PageRank
Top Users
Repeat

图处理流水线

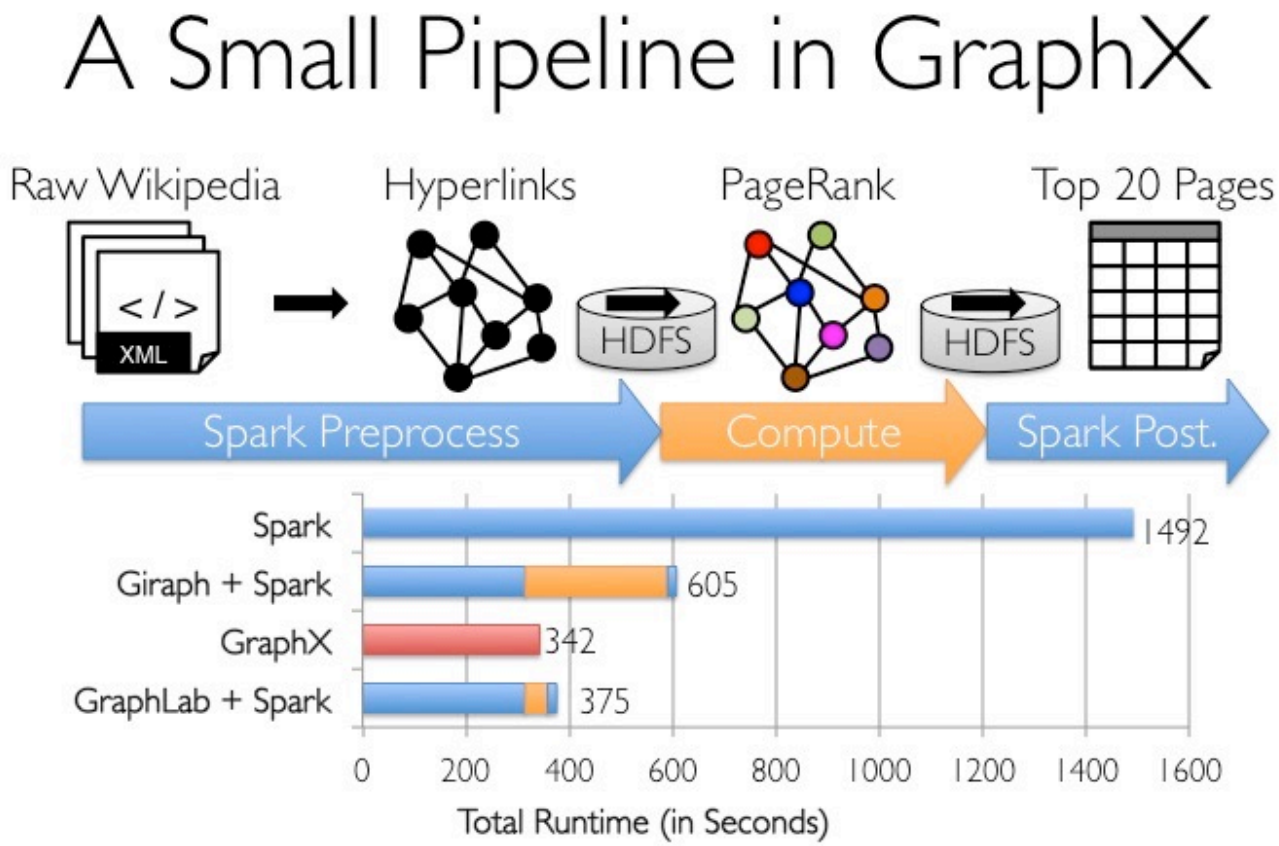
Inefficient

Extensive data movement and duplication across the network and file system



Limited reuse internal data-structures across stages

图处理流水线



Timed end-to-end GraphX is *faster* than GraphLab



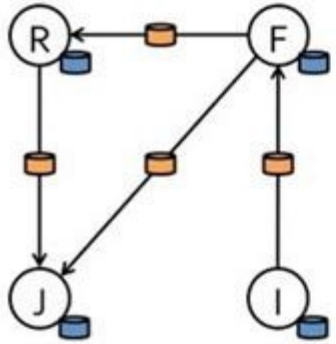
两种视图

- 对Graph视图的所有操作，最终都会转换成其关联的Table视图的RDD操作来完成。这样对一个图的计算，最终在逻辑上，等价于一系列RDD的转换过程。因此，Graph最终具备了RDD的3个关键特性：Immutable、Distributed和Fault-Tolerant，其中最关键的是Immutable（不变性）。逻辑上，所有图的转换和操作都产生了一个新图；物理上，GraphX会有一定程度的不变顶点和边的复用优化，对用户透明。

- 两种视图底层共用的物理数据，由RDD[VertexPartition]和RDD[EdgePartition]这两个RDD组成。点和边实际都不是以表Collection[tuple]的形式存储的，而是由VertexPartition/EdgePartition在内部存储一个带索引结构的分片数据块，以加速不同视图下的遍历速度。不变的索引结构在RDD转换过程中是共用的，降低了计算和存储开销。

两种视图

Property Graph



Vertex Property Table

Id	Property (V)
Rxin	(Stu., Berk.)
Jegonzal	(PstDoc, Berk.)
Franklin	(Prof., Berk)
Istoica	(Prof., Berk)

Edge Property Table

SrcId	DstId	Property (E)
rxin	jegonzal	Friend
franklin	rxin	Advisor
istoica	franklin	Coworker
franklin	jegonzal	PI

- Table视图将图看成Vertex Property Table和Edge Property Table等的组合，这些Table继承了Spark RDD的API(filter, map等)。
- Graph视图上包括reverse/subgraph/mapV(E)/joinV(E)/mrTriplets等操作。



GraphX编程

- 属性图是一个用户定义顶点和边的有向多重图。
- 有向多重图是一个有向图，它可能有多个平行边共享相同的源顶点和目标顶点。
- 多重图支持并行边的能力简化了有多重关系的建模场景。每个顶点是由具有64位长度的唯一标识符（VertexID）作为主键。GraphX没有对顶点添加任何顺序的约束。同样，每条边具有相应的源顶点和目标顶点的标识符。
- 属性表的参数由顶点（VD）和边（ED）的类型决定。



GraphX编程

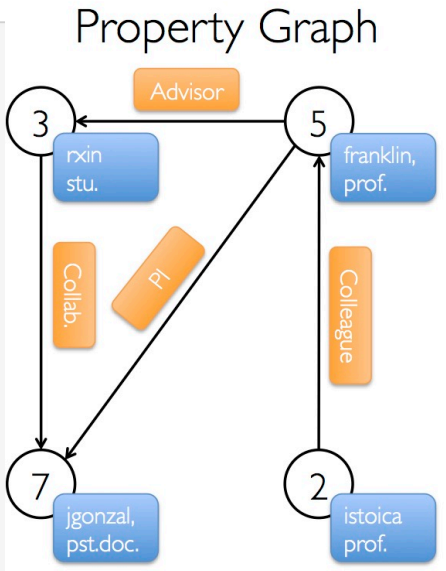

```scala
class VertexProperty()
case class UserProperty(val name: String) extends VertexProperty
case class ProductProperty(val name: String, val price: Double) extends VertexProperty
// The graph might then have the type:
var graph: Graph[VertexProperty, String] = null
```

```scala
class Graph[VD, ED] {
  val vertices: VertexRDD[VD]
  val edges: EdgeRDD[ED]
}
```

GraphX的图操作

- 构造图
  - 通过Graph Object构造
  - 通过Graph Builder构造

```scala
// Assume the SparkContext has already been constructed
val sc: SparkContext
// Create an RDD for the vertices
val users: RDD[(VertexId, (String, String))] =
  sc.parallelize(Array((3L, ("rxin", "student")), (7L, ("jgonzal", "postdoc")),
                       (5L, ("franklin", "prof")), (2L, ("istoica", "prof"))))
// Create an RDD for edges
val relationships: RDD[Edge[String]] =
  sc.parallelize(Array(Edge(3L, 7L, "collab"),    Edge(5L, 3L, "advisor"),
                       Edge(2L, 5L, "colleague"), Edge(5L, 7L, "pi")))
// Define a default user in case there are relationship with missing user
val defaultUser = ("John Doe", "Missing")
// Build the initial Graph
val graph = Graph(users, relationships, defaultUser)
```



Vertex Table

Id	Property (V)
3	(rxin, student)
7	(jgonzal, postdoc)
5	(franklin, professor)
2	(istoica, professor)

Edge Table

SrcId	DstId	Property (E)
3	7	Collaborator
5	3	Advisor
2	5	Colleague
5	7	PI



GraphX的图操作

- GraphLoader.edgeListFile提供了一种从磁盘上边的列表载入图的方式。

```
object GraphLoader {
  def edgeListFile(
      sc: SparkContext,
      path: String,
      canonicalOrientation: Boolean = false,
      minEdgePartitions: Int = 1)
    : Graph[Int, Int]
}
```

```
import org.apache.spark.graphx.GraphLoader

// Load the edges as a graph
val graph = GraphLoader.edgeListFile(sc, "data/graphx/followers.txt")
```

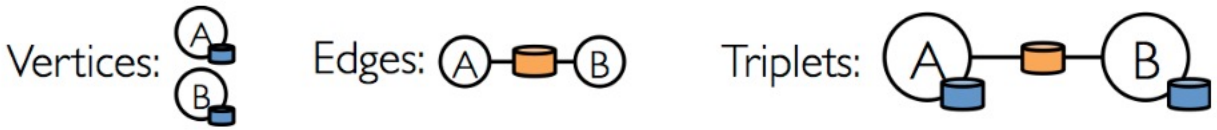


GraphX的图操作


```scala
val graph: Graph[(String, String), String] // Constructed from above
// Count all users which are postdocs
graph.vertices.filter { case (id, (name, pos)) => pos == "postdoc" }.count
// Count all the edges where src > dst
graph.edges.filter(e => e.srcId > e.dstId).count
```

```scala
graph.edges.filter { case Edge(src, dst, prop) => src > dst }.count
```

```sql
SELECT src.id, dst.id, src.attr, e.attr, dst.attr
FROM edges AS e LEFT JOIN vertices AS src, vertices AS dst
ON e.srcId = src.Id AND e.dstId = dst.Id
```





GraphX的图操作

- 属性操作
- 转换操作
- 结构操作
- 关联操作
- 聚合操作
- 缓存操作



Table Operators

- Table (RDD) operators are inherited from Spark:

```
map             reduce          sample
filter          count           take
groupBy         fold            first
sort            reduceByKey     partitionBy
union           groupByKey      mapWith
join            cogroup         pipe
leftOuterJoin   cross           save
rightOuterJoin  zip             ...
```



Graph Operators


```scala
/** Summary of the functionality in the property graph */
class Graph[VD, ED] {
  // Information about the Graph ===================================================================
  val numEdges: Long
  val numVertices: Long
  val inDegrees: VertexRDD[Int]
  val outDegrees: VertexRDD[Int]
  val degrees: VertexRDD[Int]
  // Views of the graph as collections =============================================================
  val vertices: VertexRDD[VD]
  val edges: EdgeRDD[ED]
  val triplets: RDD[EdgeTriplet[VD, ED]]
  // Functions for caching graphs ==================================================================
  def persist(newLevel: StorageLevel = StorageLevel.MEMORY_ONLY): Graph[VD, ED]
  def cache(): Graph[VD, ED]
  def unpersistVertices(blocking: Boolean = true): Graph[VD, ED]
  // Change the partitioning heuristic  ============================================================
  def partitionBy(partitionStrategy: PartitionStrategy): Graph[VD, ED]
  // Transform vertex and edge attributes ==========================================================
  def mapVertices[VD2](map: (VertexId, VD) => VD2): Graph[VD2, ED]
  def mapEdges[ED2](map: Edge[ED] => ED2): Graph[VD, ED2]
  def mapEdges[ED2](map: (PartitionID, Iterator[Edge[ED]]) => Iterator[ED2]): Graph[VD, ED2]
  def mapTriplets[ED2](map: EdgeTriplet[VD, ED] => ED2): Graph[VD, ED2]
  def mapTriplets[ED2](map: (PartitionID, Iterator[EdgeTriplet[VD, ED]]) => Iterator[ED2])
    : Graph[VD, ED2]
```



Graph Operators


```
// Modify the graph structure ================================================================
def reverse: Graph[VD, ED]
def subgraph(
    epred: EdgeTriplet[VD,ED] => Boolean = (x => true),
    vpred: (VertexId, VD) => Boolean = ((v, d) => true))
  : Graph[VD, ED]
def mask[VD2, ED2](other: Graph[VD2, ED2]): Graph[VD, ED]
def groupEdges(merge: (ED, ED) => ED): Graph[VD, ED]
// Join RDDs with the graph ==================================================================
def joinVertices[U](table: RDD[(VertexId, U)])(mapFunc: (VertexId, VD, U) => VD): Graph[VD, ED]
def outerJoinVertices[U, VD2](other: RDD[(VertexId, U)])
    (mapFunc: (VertexId, VD, Option[U]) => VD2)
  : Graph[VD2, ED]
// Aggregate information about adjacent triplets =============================================
def collectNeighborIds(edgeDirection: EdgeDirection): VertexRDD[Array[VertexId]]
def collectNeighbors(edgeDirection: EdgeDirection): VertexRDD[Array[(VertexId, VD)]]
def aggregateMessages[Msg: ClassTag](
    sendMsg: EdgeContext[VD, ED, Msg] => Unit,
    mergeMsg: (Msg, Msg) => Msg,
    tripletFields: TripletFields = TripletFields.All)
  : VertexRDD[A]
```



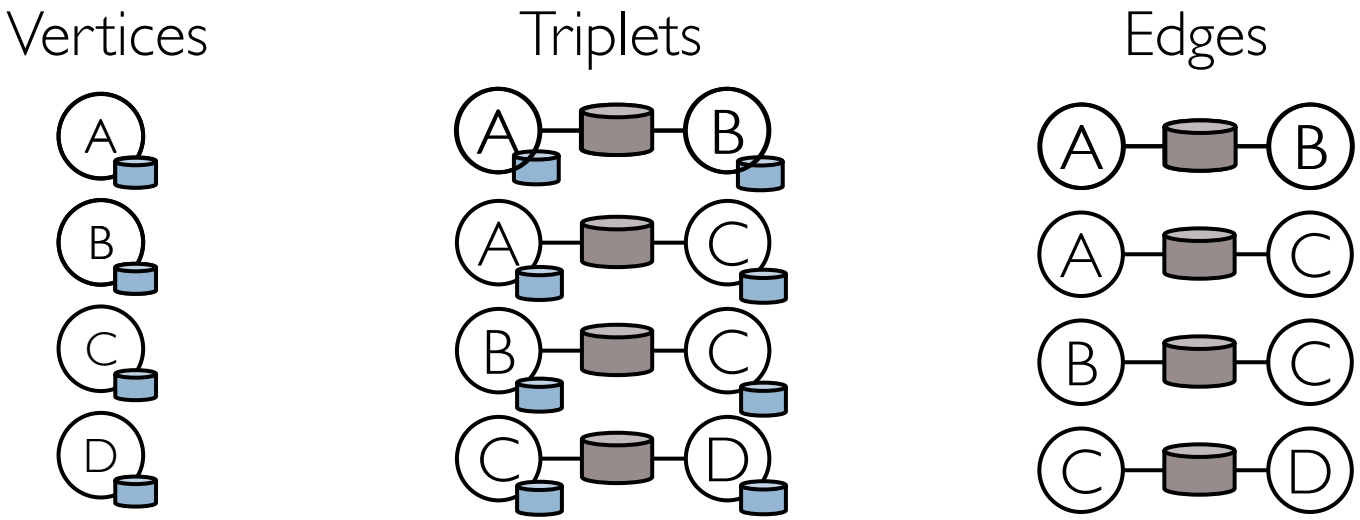
Graph Operators


```
  // Iterative graph-parallel computation ========================================================
  def pregel[A](initialMsg: A, maxIterations: Int, activeDirection: EdgeDirection)(
      vprog: (VertexId, VD, A) => VD,
      sendMsg: EdgeTriplet[VD, ED] => Iterator[(VertexId,A)],
      mergeMsg: (A, A) => A)
    : Graph[VD, ED]
  // Basic graph algorithms ======================================================================
  def pageRank(tol: Double, resetProb: Double = 0.15): Graph[Double, Double]
  def connectedComponents(): Graph[VertexId, ED]
  def triangleCount(): Graph[Int, ED]
  def stronglyConnectedComponents(numIter: Int): Graph[VertexId, ED]
}
```



Triplets Join Vertices and Edges

- The ***triplets*** operator joins vertices and edges:




```
val graph: Graph[(String, String), String] // Constructed from above
// Use the triplets view to create an RDD of facts.
val facts: RDD[String] =
  graph.triplets.map(triplet =>
    triplet.srcAttr._1 + " is the " + triplet.attr + " of " + triplet.dstAttr._1)
facts.collect.foreach(println(_))
```



常用图算法

- PageRank算法
- 三角形计数算法
- 连接分量算法



PageRank算法

- GraphX自带PageRank的静态和动态实现，放在PageRank对象中。静态的PageRank运行固定数量的迭代，而动态的PageRank运行直到排名收敛。

```
import org.apache.spark.graphx.GraphLoader

// Load the edges as a graph
val graph = GraphLoader.edgeListFile(sc, "data/graphx/followers.txt")
// Run PageRank
val ranks = graph.pageRank(0.0001).vertices
// Join the ranks with the usernames
val users = sc.textFile("data/graphx/users.txt").map { line =>
  val fields = line.split(",")
  (fields(0).toLong, fields(1))
}
val ranksByUsername = users.join(ranks).map {
  case (id, (username, rank)) => (username, rank)
}
// Print the result
println(ranksByUsername.collect().mkString("\n"))
```



三角形计数算法

- 计算通过各顶点的三角形数目，从而提供集群的度。TriangleCount要求边的指向（srcId<dstId），并使用Graph.partitionBy分割图形。

```scala
import org.apache.spark.graphx.{GraphLoader, PartitionStrategy}

// Load the edges in canonical order and partition the graph for triangle count
val graph = GraphLoader.edgeListFile(sc, "data/graphx/followers.txt", true)
  .partitionBy(PartitionStrategy.RandomVertexCut)
// Find the triangle count for each vertex
val triCounts = graph.triangleCount().vertices
// Join the triangle counts with the usernames
val users = sc.textFile("data/graphx/users.txt").map { line =>
  val fields = line.split(",")
  (fields(0).toLong, fields(1))
}
val triCountByUsername = users.join(triCounts).map { case (id, (username, tc)) =>
  (username, tc)
}
// Print the result
println(triCountByUsername.collect().mkString("\n"))
```



连接分量算法

- 连接分量算法标出了图中编号最低的顶点所连接的子集。

```
import org.apache.spark.graphx.GraphLoader

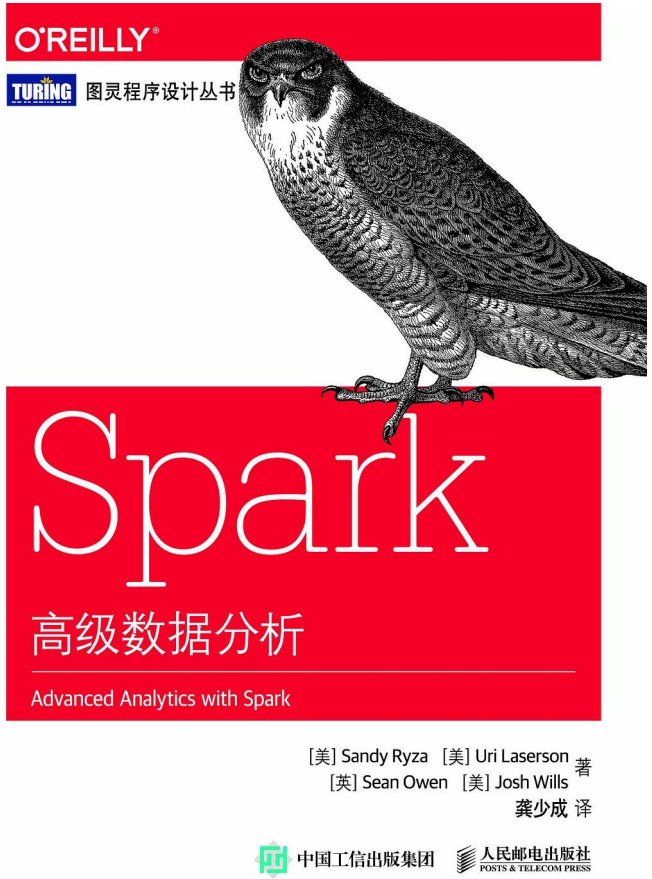
// Load the graph as in the PageRank example
val graph = GraphLoader.edgeListFile(sc, "data/graphx/followers.txt")
// Find the connected components
val cc = graph.connectedComponents().vertices
// Join the connected components with the usernames
val users = sc.textFile("data/graphx/users.txt").map { line =>
  val fields = line.split(",")
  (fields(0).toLong, fields(1))
}
val ccByUsername = users.join(cc).map {
  case (id, (username, cc)) => (username, cc)
}
// Print the result
println(ccByUsername.collect().mkString("\n"))
```



应用场景

- 图谱体检平台
- 多图合并工具
- 能量传播模型
- ……

推荐





https://github.com/sryza/aas

THANK YOU



南京大学
NANJING UNIVERSITY

南京大学计算机软件研究所
Institute of Computer Software, Nanjing University